

Semi-Automatic Generation of Extended Finite State Machines from Natural Language Standard Documents

Juliana Galvani Greghi*[†]
 *Computer Science Department
 University of Lavras
 Lavras, M.G., Brazil
 Email: juliana@dcc.ufla.br

Eliane Martins[†], Ariadne M. B. R. Carvalho[†]
[†]Institute of Computing
 University of Campinas
 Campinas, S.P., Brazil
 Email: {eliane, ariadne}@ic.unicamp.br

Abstract—Many requirement documents are written in natural language and, therefore, may contain problems such as inconsistencies and ambiguities. To minimize these problems, there is a trend in Software Engineering to use models to represent systems. These models are obtained from textual requirements. However, manual modeling is a complex task and, in order to do it semi-automatically, one has to deal with problems such as the kind of model to be generated, the automation degree to be achieved, and the quality of the document that must be processed. We propose a methodology to semi-automatically generate Extended Finite State Machines (EFSMs) from natural language standard documents. We used Natural Language Processing (NLP) techniques and tools to extract information from the document, and implemented a prototype which generates EFSMs. The generated EFSMs were validated with a model checking tool, and manually evaluated by comparing them with the manually generated models

Keywords: *Natural Language Processing, Model Generation, Aerospace Domain, Extended Finite State Machines.*

I. INTRODUCTION

The transformation of requirements into models has motivated many researchers over the years. There is no manual or guideline to indicate how to perform this transformation. The quality of the document has a direct influence on the transformation process [1]. Meyer [2] and Firesmith [3] identified a set of common problems in requirements document, such as ambiguity, inconsistency, and the use of inappropriate vocabulary, which may increase the development cost and cause a lot of rework. Mich, Franch, and Inverardi [4] showed that the majority of the requirements documents is written in natural language. This suggests that requirement analysis could strongly benefit from NLP-based support systems [4]. In this context, several researchers have proposed approaches to improve the quality of the requirements document, and to extract information used to generate formal or semi-formal models, therefore improving the entire development process. Many of these approaches use a set of tools and NLP techniques to pre-process and extract information from the documents [5][6][7]. Text pre-processing usually relies on a syntactic analyzer (or parser) and/or a Part-of-Speech (PoS) tagger. The parser provides a valid syntactic structure for a sentence. The process is supported by a grammar that defines a set of valid structures for a given domain [8][9]. The PoS tagger associates

syntactic and morphological features to the words of a sentence or text [8]. In Figure 1, we show a sample sentence, processed by the Stanford Parser [9] and Stanford Tagger [10].

Sentence: *The system sent many error codes.*
Parsing result
 (ROOT (S (NP (DT The) (NN system)) (VP (VBD sent)
 (NP (JJ many) (NN error) (NNS codes))) (. .)))
Tagging result
 The_DT system_NN sent_VBD many_JJ error_NN codes_NNS ._-

Fig. 1. Sentence processed by the Stanford Parser and the Stanford Tagger

The result from the parser shows that the sentence is composed of a noun phrase (NP) and a verb phrase (VP). The head of the NP is the noun “system”, and the head of the VP is the verb, in past tense, “sent”. The result from the tagger shows the lexical categories attributed to each word of the sentence, e.g., “sent” is a verb, in the past tense (tag `_VBD`), and “many” is an adjective (tag `_JJ`). This kind of information can be very helpful in generating different kinds of models.

Results from the pre-processing are used to obtain structured information from the text. In our case, the text is a requirements document, and the structured information are models representing the requirements. A system model is an abstraction of the system under consideration, and it is expected to be less prone to ambiguities and inconsistencies than a natural language description. Moreover, it can be more efficient in the communication with the development team. There are different kinds of system models that can be generated from requirement documents. Our focus is on the extraction of system behavior models from natural language requirements. The extracted behavior model is represented by an Extended Finite State Machine (EFSM).

EFSMs are usually used to deal with more complex systems, such as applications in the aerospace domain. They can represent different layers of abstraction [11], and are very useful for modeling applications in which it is necessary to represent control and data information, as it is the case of communication services for space missions. Furthermore, an EFSM can drive specific tools, providing an efficient way to test the system, as well as an easier way to detect errors or omissions in the requirements description [12].

Requirements can be documented in various forms. For some applications, requirements are described in a standard document, e.g., communication protocols, automotive systems, and on-board services. We are interested in standard documents because they are the basis for many implementations. Besides, they go through less frequent changes than ordinary requirement documents. Although problems such as ambiguity and incompleteness appear less frequently in standard documents, other issues must be addressed, such as the possibility that relevant information for the same element could be distributed over different sections of the document, or that important information be presented on tables or figures, among others. Information extraction for model generation must deal with the correct identification of atomic and relational information. A major difficulty in this task is dealing with ambiguity; some approaches propose rewriting techniques to eliminate ambiguities in the text. Other problem is nontextual information, expressed in the form of tables or figures. This is a problem for automatic model generation, specially the behavioral model, because essential facts for behavior specification are omitted. Although there are many approaches to extract information from these data, many of them only apply to web documents and marked text [13][14][15]. There are few approaches to treat unmarked texts (e.g. “.pdf”) [16]. The presence of relevant information in tables and figures makes NLP pre-processing more difficult and, considering the complexity that could be involved, in this work we have manually identified and stored this information in a database, to be used during EFSM generation.

This work presents a methodology to semi-automatically transform a standard document in natural language into Extended Finite State Machines. The information used for EFSM generation is extracted from a standard document with NLP tools and techniques; structured documents are specially treated, with explicit use of the structure in this phase. The complex problem of extracting information from tables and figures is finessed, through manual identification and storing it in a database used during EFSM generation.

A standard document, developed by the European Cooperation for Space Standardization (ECSS), a cooperation between the European Space Agency (ESA) and aerospace industries [17], was used in a case study. The standard document “Space Engineering - Ground Systems And Operations - Telemetry And Telecommand Packet Utilization Service”, called PUS, and proposed by ECSS, describes a set of services for spacecraft monitoring, and can be used in different missions [17]. The National Institute for Space Research (Instituto Nacional de Pesquisas Espaciais - INPE) in Brazil, uses this standard document in some of its projects. We had the collaboration of researchers from INPE and from the Aeronautics Technology Institute (ITA) to develop a case study whose aim was to evaluate the proposed methodology.

This paper is organized as follows: Section 2 presents the background, Section 3 introduces the proposed methodology for information extraction, and in Section 4 the work is evaluated with a case study and conclusions and future work are presented in Section 5.

II. BACKGROUND

The transformation of requirement documents into formal representations has been explored along the years, and the

efforts range from improving the quality of the document to the kind of model to be generated. Some researchers convert text requirements to mathematical or logical formulae, and others convert text requirements to models, mainly UML models.

Yue, Briand and Labiche [1], present a systematic review in which they described several approaches which propose the use of different concepts and techniques to perform this transformation. The authors defined taxonomies to classify requirements, restriction rules, analysis models, and transformation approaches. The available approaches may or may not use domain specific information as part of the input document to aid information extraction; also, the requirements document may or not be restricted to a specific format, and so do the allowed vocabulary and sentence structure. Some approaches provide a template to represent requirements in a structured format. The most common is the use case representation. The vocabulary and sentence structure can be restricted to reduce problems and the complexity of the documents. This makes the transformation process more efficient and reliable. However, such restrictions vary from one approach to another. Regarding the analysis models, the authors found that many approaches generate objects, classes and abstractions, and some of them also generate behavioral features. UML is the most common language used to represent the analysis model. Regarding the automation aspect, the same authors found that most of the approaches are automated or semi-automated. Nevertheless, they have not found an efficient and sufficiently documented approach to generate analysis models, either automatically or semi-automatically. Most reviewed approaches use at least one pre-processing technique. PoS tagging and syntactic analysis are largely used to facilitate transformation and to determine the grammatical structure of the sentence. Also, most approaches use at least one intermediate model. The transformation processes identified by the authors are: rule-based, pattern-based, ontology-based, and identity-based approaches. The rule-based approach is the most common, and uses a pre-defined set of rules to transform requirements into analysis models; an ontology-based approach uses an ontology as an intermediate model that is transformed into an analysis model; pattern-based analysis looks for patterns, transforming them into a target pattern; and, finally, the identity-based approach transforms a model into another representation, without changing the information content.

In addition to the approaches analyzed in [1], Yue, Briand and Labiche [5][18] present an approach to converting natural language use cases to UML models (class, activity, and state diagrams). It was implemented as part of a tool, called aToucan. Transformation algorithms and NLP tools were used to extract information and to convert it to UML analysis models. Use cases are specified according to a use case modeling approach (RUCM) [19] based on a template, and on rules for Use Case Specification (UCS). It is an automatic rule-based approach that uses as input use cases written according to a pre-defined format, and pre-process the text using a syntactic analyser, generating an intermediate model.

Deeptimahanti and Babar [20] transform requirement documents into UML models (class, use cases, and collaboration diagrams), implemented with a tool called UMGAR. The tool contains two components: the normalizing requirements component and the model generator component. The process

initiates with the normalizing process, i.e. requirements are rewritten to remove ambiguities from the sentences. After the text has been normalized, the model generator extracts the information needed to generate the analysis model. The model can be converted to XMI files, allowing requirements traceability [7]. It is a semi-automatic hybrid approach (rule-based and identity-based), which takes natural language requirements as input, pre-processes the text using a syntactic analyser and other NLP tools to resolve ambiguities and other inconsistency problems in the rewriting process, and generates intermediate models.

Kof [6] presents an approach to convert textual descriptions of an automaton into a finite automaton model. The identification of states is based on specific word categories and terms defined by the user. The identification of transitions is based on the segmentation of the sentences, and on the categorization of the sentence segments. A set of rules were defined to find the text segments that should be used. The generated automaton is represented in a tabular form. It is an automatic rule-based approach which uses a textual description of an automaton as input and pre-process the text using a PoS Tagger.

The approach proposed here is semi-automatic, rule-based, and uses a PoS tagger to pre-process the text. It takes a standard document as input, and generates Extended Finite State Machines (EFSMs). Differently from [5][18], the input is not restricted to a specific format, and there is not a normalizing process, as in [20][7]. The search for patterns is similar to [6], but in addition, our approach uses structural patterns from the standard document to extract information which will be used for model generation.

III. METHODOLOGY

Our approach was inspired by Kof's [6]. However, in addition to the information from the PoS tagger, the approach proposed here uses the structure of the document as help to identify the relevant information to be extracted, and which will be used to compose the models. It is important to note that the writing style and the structure of the document are very relevant; the information description, as well as the structure of the document, must follow some patterns. The proposed approach uses the structural patterns of the sentences and of the document to identify and extract information. The information extraction process presents fewer problems than those reported in the literature: a standard document is usually organized in chapters and/or sections, and the description of the element that must be modeled (e.g. a service or a procedure for a specific application) may refer to other sections (or chapters) of the same document, or to other documents. In some domains, the document may describe the terms and abbreviations, and other relevant information that might be used in the implementation of any element described in the standard document. To illustrate the methodology we present a sample text (Figure 2) showing some relevant characteristics present in a standard document, together with some problems identified and treated in this work.

Step 1: Semi-automated analysis of the text

The first activity in the proposed methodology is the semi-automated analysis of the standard document. The analysis

This is a fictitious standard document to illustrate the proposed information extraction methodology.

Standard document of network communication protocol Chapter 3 - Data transfer protocol

3.1 Description: This chapter defines a protocol for data transfer. Data are organized in packets, as described in chapter 1. When a packet is sent, a monitoring activity is initiated.

3.2 Concepts: The data transfer functionality occurs in three stages:

a) Verification of data packets: the data must be divided into small pieces. The organization of the packets must be verified to avoid corrupted packets.
b) Transference of data packets: different delivery mechanisms must be used to avoid packet loss. An error shall occur if there are packets delivered out of order.

c) Verification of data integrity: all packets must be verified. If no transferred packet was lost, a success message must be transmitted to the sender.

3.3 Available functionalities

3.3.1 Basic functionalities

Verification of data packets (1,1)

Transferring of packets (2,1)

Verification of data packets integrity (3,1)

In addition of the description, other details about the protocol are described in the last chapters of this manual. To details consult Appendix A- HowTo.

3.3.2 Additional functionalities

Sending a warning e-mail (4,2)

1.4 Dependencies

ID	Name	Requires
1	Verification of data packing	
2	Transference of packets	
3	Verification of data integrity	2

Fig. 2. Text fragment used to illustrate the proposed methodology

starts with a word frequency counting. The analysis of sentences containing the most frequent words may help to identify the patterns of the sentences. It is important to note that stop words, i.e., those words that have a high frequency but no semantical meaning [21] are not taken into account in the analysis. An example of an identified pattern is: most of the sentences initiating with the word "when" present a condition for the execution of an action, described in the sequence of the sentence - "When a packet is sent, a monitoring activity is initiated.". This is an example of a term to be used in the identification of actions and conditions.

Step 2: Reading of the text

After finding patterns in the sentences, the next activity is the careful manual reading of the entire text. References to other documents or to a different section of the text must be marked for future processing. Information represented in tables or figures that are not explicit in the text, and are related to the described element, must also be marked and processed in a later stage. In Figure 2, the dependency relation of the functionalities is described only in the table and must be marked to be used in later stages of the process.

Step 3: Reading of the element description

The analysis of the section/chapter that will be used for model generation must be performed to better understand the structure of the section and relations between information. The text must be pre-processed using a PoS Tagger, which will help the analyst to identify word categories that can be used in the information extraction process. Each category attributed to a word is a tag, and can be used in the extraction process. In addition to the categories, the structural pattern identified in the document can also be used for information extraction.

Step 4: Identification of states

The identification of states can rely on the location of the information, e.g., the section of the document where the information is described, and on the sequence of the tags. In the example, we can see that the stages of the operation described in the text of Figure 2 are located in a section, identified by the number of the chapter, and by the number of the section. This information can be described as the pattern below, where `_CD` is the tag used by the PoS Tagger to identify cardinal numbers:

`<chapter number> . <section number>_CD`

By analyzing the text fragments, it is possible to identify that all stages end with the expression “of data” followed by a noun (tags initiated with `_NN`). They are composed of a noun, followed by the expression “of data” plus a noun. The search pattern can be described as below. The search_expression must be replaced, in the example, by the expression “of_IN data_NNS”.

```
<s>::= <a><b><c>
<a>::= “_nn” | “_nns” | “_nnp”
<b>::= “search_expression”
<c>::= “_nn”<c> | “_nns”<c> | “_nnp”<c> | <d>
<d>::= ε
```

Figure 3 shows a fragment of the text presented in Figure 2, processed by the Stanford PoS Tagger [10]. In Figure 3 it is possible to verify the pattern described above in the underlined text segments. Another important observation is that the order in which the states are described can be used to define the source and destination states from each transition, e.g., events occurred in the state “Verification of data packets” cause a transition to the state “Transference of data packets”.

a_DT	-RRB-	-RRB-	Verification_NN	of_IN	data_NNS	packets_NNS:
the_DT	data_NNS	must_MD	be_VB	divided_VBN	into_IN	(...)
b_LS	-RRB-	-RRB-	Transference_NN	of_IN	data_NNS	packets_NNS:
different_JJ	delivery_NN	mechanisms_NNS	must_MD	(...)		

Fig. 3. Text segments to state identification

Step 5: Identification of transitions

A transition is the change from a state to another, according to a set of events or conditions being satisfied or not [11], that may require an execution of an action. Identification of transitions proceeds very similarly to identification of states, starting with the analysis of the text. In the text fragment presented in Figure 2, events were identified using the following pattern:

`<event_name> (<functionality_number>,<frequency>)`

The relation of the identified events with the specific element to be modeled was extracted from the pair of numbers, enclosed with parenthesis. The first number relates the event with the functionality, and the second indicates if it is a mandatory or optional functionality. To identify the source state of the transitional flow, the event name (indicated by the text segment before the parenthesis) will be searched in the text description of each identified state. If the event name is found in the state name or in the state description, this state is the source of the transitional flow and the destination is the next state identified in the text. A transition is also composed

by actions and conditions. Considering item 3.2.a in Figure 2, the text segment “the data must be divided into small pieces.” is an action to be performed. In the sample text, it is possible to observe that the text fragments “different delivery mechanisms must be used to avoid packet loss.” (item 3.2.b), and “all packets must be verified.” (item 3.2.c) are both composed of a determiner (tag `_DT`) or an adjective (tag `_JJ`), followed by any number of nouns (tags initiated with `_NN`) - “the data”, “different delivery mechanisms” and “all packets”. Each one is the subject of the action. The rest of the text fragment is the modal verb “must”, indicating the obligation of the action, the verb “to be” and, finally, a verb, in past participle (tag `_VBN`), expressing the action - “divided”, “used” and “verified”. The sentences searched for must satisfy the following description:

```
<s>::= <a><b>
<a>::= “_dt” <c> | “_jj” <c>
<b>::= “must_MD be_VB” <d>
<c>::= “_nn” | “_nns” | “_nnp” | <c>
<d>::= “_vbn”
```

The last activity is the identification of the conditions, also known as guards. In general, guards are conditions that must be satisfied to enable the transition from a state to another. In most cases, sentences that express conditions contain the conjunction “if” and the condition is expressed after the conjunction. It is not rare that sentences expressing conditions also define the actions to be executed in case these conditions are satisfied.

The identification of conditions starts of searching for sentences containing the conjunction “if”. The text fragment after the conjunction is a condition. If the text segment after the conjunction is divided by a comma, the text segment after the comma is the action directly related to the satisfaction of the condition. To illustrate this, consider item 3.2.c. in Figure 2: the second sentence starts with the conjunction “if” and the following text segment, until the comma, is the condition - “no packet transferred was lost”. After the comma, the segment represents the action to be executed - “a success message must be transmitted to the sender.”. Another way to identify conditions is to search for exceptional conditions, i.e., conditions that, if satisfied, must generate an error. In this case, the exceptional action must be searched before the conjunction “if”, and the condition is defined after the conjunction. The difference is that the sentence ends with the definition of the condition. Consider the item 1.2.b in Figure 2: the second sentence starts with the expression “An error shall occur” which is the action, and can be used as a key expression in the information extraction process to identify exceptional conditions. After the conjunction “if”, the subsequent text is the condition to be satisfied - “there are packets delivered out of order”. In the identification of conditions and actions there are not sequences of tags to be searched, only key expressions. The marks used in the search process are the conjunction “if”, the division of the sentence by a comma, and the exceptional condition that starts with a key expression.

It is important to say that Kof [6] splits the text sentences in categories and uses these categories to identify transitions and their parameters. We do not use these categories because they

were not suitable for the texts which we have analyzed. The sentence structure of the standard document is very different from the textual description of an automaton, as the one analyzed in [6]. The use of structural patterns, e.g the location of specific information, or the order of information items, combined with the definition of terms and patterns of tags to be searched, were sufficient to solve the problem. It is important to note that automatically extracted information, as in [6], must be manually revised at a later stage of the process before it can be used in the development process.

IV. EVALUATION

The proposed methodology was evaluated with the PUS standard document case study. The developed prototype generated the EFSM for two selected services. The document specifies requirements of use of telecommand packets and telemetry source packets for monitoring and controlling subsystems and payloads [17]. It is applicable to any mission, and it takes into account that different missions require different levels of complexity and functionality. No service is mandatory, but each one has a set of minimum capabilities that must be always included in any implementation of a service, and a set of additional ones [17]. With this case study we answered the following research questions: a) Are the automatically extracted EFSM similar to those manually created? b) Are the automatically generated EFSM free from deadlocks¹? c) Do the automatically generated EFSM satisfy the properties of reachability and liveness?

The evaluation of the generated EFSMs was performed in two ways: 1) A manual analysis, comparing the automatically generated EFSM with a manually created EFSM, available in V  ras [22]; 2) A model checking using the DiVinE Model Checker [23], which verifies the presence of accepted cycles, the reachability, the presence of deadlocks, and assertion violation.

The first comparison showed some differences between the resulting state machines for Telecommand Verification service; as it can be seen in Figure 4, the automatically generated state machine starts in the state “Acceptance”, while the manually created state machine (Fig. 5) starts in “Without TC”.

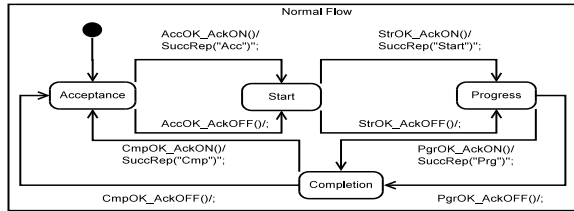


Fig. 4. Representation of automatically generated EFSM

Although this state is not described in the text, the expert users understand that the initial state of a service must always be “empty” or “Without a TC”. In this context, we have introduced this initial state in our state machines as a default state. The “Checking TC” state, presented in Figure 5, is represented in the automatically generated state machine as

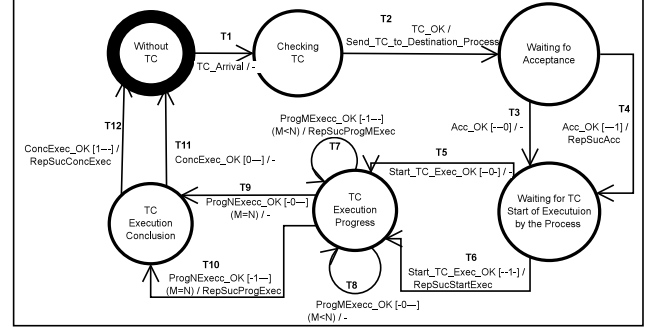


Fig. 5. Manually created EFSM. Adapted from [22]

guard conditions. These conditions are text segments in natural language, which were not presented in Figure 4 due to space limitations. Another difference is that self loops (Figure 5- T7, T8) are difficult to be treated, but this would be possible if there was a clear reference to the stage affected by an event. However, if this information is not explicit in the text, the identification of self loops could be highly dependent on human interpretation, or it would require semantic analysis, which is beyond the capabilities of the tools used. The last difference regards the information stored in the database, for example, the Ack field. In the manually created state machine, it is represented by curly brackets filled up with hyphens and “1”. In the automatically generated state machine, this information is expressed textually (“AckON” or “AckOFF”).

Despite the differences, it is possible to see that the main information about the states and transitions are the same; this is a strong indication that the automatically generated state machine is very close to that manually created, answering affirmatively the first research question posed at the beginning of this section. In addition, the On-board operation procedure service was processed, and an EFSM for the normal and the exceptional flows was generated. However, due to the lack of a manual model for this service no comparative evaluation was possible.

The output generated by the prototype is a text file with the .sm extension. This is the extension used by the State Machine Compiler (SMC) [24], an application that allows defining state diagrams using an easy-to-understand language. It can be used to generate source code of the state machine in many programming languages, such as C and Java. SMC also generates Graphviz [25] .dot files from .sm files. Graphviz is an open source software for graph visualization. It is used in our prototype to generate a graphical view of the EFSM.

The second evaluation process was performed using the model checker tool DiVinE 3.2.2 [23]. This tool is able to verify models in a wide range of languages. The generated state machines were modeled using the .dve language, the modeling language created specifically for DiVinE; the resultant models were submitted to DiVinE to be validated for safety and some liveness properties. The generated state machines for the Telecommand Verification and On-board Operations Procedure services were validated and all properties held: no deadlocks were found in either model, and the reachability and liveness also held, thus answering affirmatively the second and third research questions posed at the beginning of this section.

¹ It is important to notice that deadlock, in this context, means a state in the system that has no legal successors

V. CONCLUSIONS AND FUTURE WORK

The transformation of natural language documents into models present many challenges, even if the text does not show the most common problems reported in the literature, such as ambiguity and incompleteness.

The transformation of standard documents into EFSM can be helpful. Nevertheless, the set of rules created for information extraction are very dependent on the structure of the document, and on the writing style. Since a standard document will be used by many people, for research, industrial or critical purposes, the considerable work involved is paid back later with increased productivity. Indeed, effort applied in the adequacy of the rules may be rewarded by the benefits that brought by a representation that can be shared by all sorts of development groups.

EFSMs can be helpful in many development phases, specially in the test phase, since it can be automatically drive appropriate tools. Still, the automatically generated EFSM is an initial representation of what is been required by the standard document, and the expert users can use this as help in the development of the final models. However, just by themselves, these models may not be sufficient for a complete analysis and, as future work, we must investigate which models could be generated on extending the methodology proposed here. As self loops, hierarchies can be treated only if the relationships are explicit in the text.

The use of structural information of the standard document in the information extraction can help the model generation, requiring less pre-processing of the text, eschewing a representation in a specific format, or any change in the sentences.

VI. ACKNOWLEDGMENT

The authors would like to thank Ana Maria Ambrósio, from the National Institute for Space Research, Emília Villani, from the Aeronautics Technology Institute, and Raphael Winckler de Bettio, from the University of Lavras, for their valuable help throughout this research.

REFERENCES

- [1] T. Yue, L. Briand, and Y. Labiche, "A systematic review of transformation approaches between user requirements and analysis models," *Requirements Engineering*, vol. 16, no. 2, pp. 75–99, 2011. [Online]. Available: <http://dx.doi.org/10.1007/s00766-010-0111-y>
- [2] B. Meyer, "On formalism in specifications," *IEEE Softw.*, vol. 2, no. 1, pp. 6–26, Jan. 1985. [Online]. Available: <http://dx.doi.org/10.1109/MS.1985.229776>
- [3] D. Firesmith, "Common requirements problems, their negative consequences, and the industry best practices to help solve them," *Journal of Object Technology*, vol. 6, no. 1, pp. 17–33, 2007.
- [4] L. Mich, M. Franch, and P. N. Inverardi, "Market research for requirements analysis using linguistic tools," *Requir. Eng.*, vol. 9, no. 1, pp. 40–56, 2004. [Online]. Available: <http://dx.doi.org/10.1007/s00766-003-0179-8>
- [5] T. Yue, L. Briand, and Y. Labiche, "An automated approach to transform use cases into activity diagrams," in *Modelling Foundations and Applications*, ser. LNCS, T. Kühne, B. Selic, M.-P. Gervais, and F. Terrier, Eds. Springer Berlin Heidelberg, 2010, vol. 6138, pp. 337–353.
- [6] L. Kof, "Translation of textual specifications to automata by means of discourse context modeling," in *Requirements Engineering: Foundation for Software Quality*, ser. Lecture Notes in Computer Science, M. Glinz and P. Heymans, Eds. Springer Berlin Heidelberg, 2009, vol. 5512, pp. 197–211.
- [7] D. K. Deeptimahanti and R. Sanyal, "Semi-automatic generation of UML models from natural language requirements," in *Proceedings of the ISEC '11*. New York, NY, USA: ACM, 2011, pp. 165–174. [Online]. Available: <http://doi.acm.org/10.1145/1953355.1953378>
- [8] E. Charniak, "Statistical techniques for natural language parsing," *AI Magazine*, vol. 18, no. 4, pp. 33–44, 1997.
- [9] D. Klein and C. D. Manning, "Accurate unlexicalized parsing," in *Meeting of the Association for Computational Linguistics*, 2003, pp. 423–430.
- [10] K. Toutanova, D. Klein, C. D. Manning, and Y. Singer, "Feature-rich part-of-speech tagging with a cyclic dependency network," in *Proceedings of the NAACL '03 - Volume 1*. Stroudsburg, PA, USA: Association for Computational Linguistics, 2003, pp. 173–180. [Online]. Available: <http://dx.doi.org/10.3115/1073445.1073478>
- [11] V. S. Alagar and K. Periyasamy, *Specification of Software Systems*. Springer London Dordrecht Heidelberg New York, 2011.
- [12] G. V. Bochmann and A. Petrenko, "Protocol testing: Review of methods and relevance for software testing," in *Proceedings of the 1994 ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA '94. New York, NY, USA: ACM, 1994, pp. 109–124.
- [13] Y. Wang and J. Hu, "A machine learning based approach for table detection on the web," in *Proceedings of the 11th International Conference on World Wide Web*, ser. WWW '02. New York, NY, USA: ACM, 2002, pp. 242–250. [Online]. Available: <http://doi.acm.org/10.1145/511446.511478>
- [14] —, "Detecting tables in HTML documents," in *Document Analysis Systems V*, ser. Lecture Notes in Computer Science, D. Lopresti, J. Hu, and R. Kashi, Eds. Springer Berlin Heidelberg, 2002, vol. 2423, pp. 249–260. [Online]. Available: http://dx.doi.org/10.1007/3-540-45869-7_29
- [15] R. Zanibbi, D. Blostein, and J. R. Cordy, "A survey of table recognition," *Document Analysis and Recognition*, vol. 7, no. 1, pp. 1–16, 2004.
- [16] Y. Liu, K. Bai, P. Mitra, and C. L. Giles, "Tableseer: Automatic table metadata extraction and searching in digital libraries," in *Proceedings of the 7th ACM/IEEE-CS Joint Conference on Digital Libraries*, ser. JCDL '07. New York, NY, USA: ACM, 2007, pp. 91–100. [Online]. Available: <http://doi.acm.org/10.1145/1255175.1255193>
- [17] ECSS, *Ground systems and operations - Telemetry and telecommand packet utilization*. ECSS-E-70-41A. ESA Publications Division, 2003.
- [18] T. Yue, S. Ali, and L. Briand, "Automated transition from use cases to UML state machines to support state-based testing," in *Modelling Foundations and Applications*, ser. Lecture Notes in Computer Science, R. France, J. Kuester, B. Bordbar, and R. F. Paige, Eds. Springer Berlin Heidelberg, 2011, vol. 6698, pp. 115–131.
- [19] T. Yue, L. Briand, and Y. Labiche, "A use case modeling approach to facilitate the transition towards analysis models: Concepts and empirical evaluation," in *Model Driven Engineering Languages and Systems*, ser. Lecture Notes in Computer Science, A. Schürr and B. Selic, Eds. Springer Berlin Heidelberg, 2009, vol. 5795, pp. 484–498.
- [20] D. K. Deeptimahanti and M. A. Babar, "An automated tool for generating UML models from natural language requirements," in *Proceedings of ASE'09*, Nov 2009, pp. 680–682.
- [21] C. D. Manning and H. Schütze, *Foundations of statistical natural language processing*. MIT press, 1999.
- [22] P. C. Vêras, "Benchmarking Software Requirements for Space Applications Documentation," Ph.D. dissertation, Aeronautics Institute of Technology, 2011.
- [23] J. Barnat, L. Brim, V. Havel, J. Havlek, J. Kriho, M. Leno, P. Rokai, V. till, and J. Weiser, "DiVinE 3.0 – An Explicit-State Model Checker for Multithreaded C & C++ Programs," in *Computer Aided Verification (CAV 2013)*, ser. LNCS, vol. 8044. Springer, 2013, pp. 863–868.
- [24] "SMC - State Machine Compiler," <http://smc.sourceforge.net/>.
- [25] "Graphviz - Graph Visualization Software," <http://www.graphviz.org>.